# Decentralized Micro-Lending System


**Muhammad Ammar Akhter** **(BSE223132)**

**Hasham Arshad** **(BSE223150)**

**Balaj Mir** **(BSE223053)**

**BS Software Engineering**

**Department of Software Engineering**

**Capital University of Science & Technology, Islamabad**

## 1. Executive Summary

The **Decentralized Micro-Lending System** is a peer-to-peer (P2P) lending platform designed to eliminate the bureaucratic delays of traditional banking. By utilizing Ethereum-based smart contracts, the system automates loan requests, interest distribution, and repayment tracking. This project demonstrates a functional Web3 application that provides financial accessibility through a transparent, immutable ledger.

## 2. Problem Statement

Traditional micro-finance is hindered by:

- **High Intermediary Costs:** Banks and third-party lenders charge significant service fees.
- **Transparency Gaps:** Lack of a public record for loan terms and repayment history.
- **Slow Processing:** Manual verification leads to delays in fund disbursement for urgent needs.
- **Centralization:** Risk of data manipulation and single points of failure.

## 3. Project Objectives

- **Automated Lending:** Develop a Solidity contract to manage the loan lifecycle (Request $\rightarrow$ Approval $\rightarrow$ Disbursement $\rightarrow$ Repayment).
- **On-Chain Logic:** Implement fixed-rate interest calculations directly within the smart contract.
- **Seamless Integration:** Connect a React-based frontend to the Ethereum Sepolia Testnet using MetaMask.
- **Security:** Utilize industry-standard libraries (OpenZeppelin) to prevent common vulnerabilities like reentrancy.

## 4. Technical Architecture & Stack

The project follows a decentralized three-tier architecture:

- **Blockchain Layer:** Solidity Smart Contracts deployed on Sepolia Testnet.
- **Frontend Layer:** React.js with Tailwind CSS for a responsive user dashboard.
- **Provider Layer:** Ethers.js and MetaMask for blockchain interaction and wallet signing.
- **Development Suite:** Hardhat for local environment simulation and contract testing.

## 5. Functional Requirements

- **User Dashboard:** Connect wallet, view balance, and track active loan status.
- **Borrower Module:** Submit loan requests specifying the amount and duration.
- **Admin/Lender Module:** Owner-only interface to review pending requests and trigger ETH transfers.
- **Repayment Engine:** Users can pay back in installments; the contract updates the remaining balance in real-time.

## 6. Methodology & Security

We are adopting an **Agile development** approach:

1. **Phase 1:** Smart Contract development and unit testing with Mocha/Chai.
2. **Phase 2:** Backend integration and event listening using Node.js.
3. **Phase 3:** Frontend UI development and MetaMask state management.
4. **Security:** We will implement Ownable and ReentrancyGuard modifiers to ensure that only the admin can approve loans and that funds cannot be drained via recursive calls.

## 7. Future Scope

Post-semester, the system can be scaled by adding:

- **Collateralization:** Requiring ERC-20 tokens as security for loans.
- **Reputation System:** Lower interest rates for users with a history of on-time payments.
- **Lending Pools:** Allowing multiple liquidity providers to earn interest.

## 8. References

1. Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.
2. Buterin, V. (2014). Ethereum Whitepaper.
3. OpenZeppelin. (2024). Standard Contract Libraries Documentation.
4. Hardhat. (2024). Ethereum Development Environment Documentation.
5. Aave & Compound. (2023). Decentralized Lending Protocols Whitepapers.